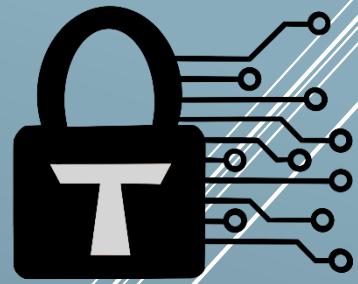


# Trust Security

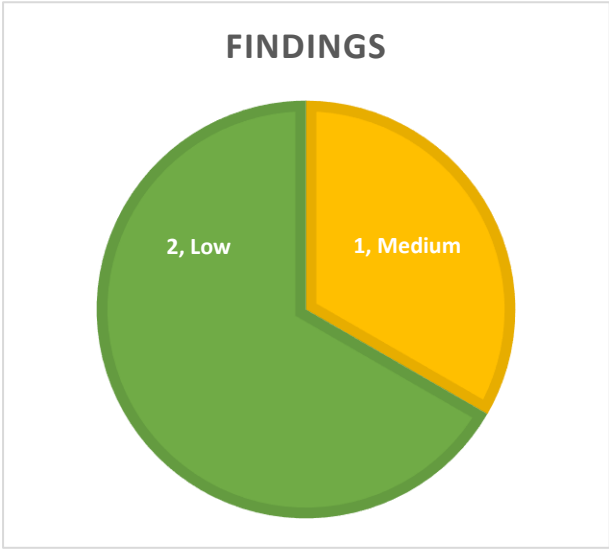


Smart Contract Audit

Curvance Final Audit

23/11/2025

# Executive summary



|                    |                                       |
|--------------------|---------------------------------------|
| Category           | Lending                               |
| Audited file count | 42                                    |
| Auditor            | Trust<br>HollaDieWaldfee<br>ether_sky |
| Time period        | 12/11/2025 - 22/11/2025               |

Findings

| Severity | Total | Fixed | Acknowledged |
|----------|-------|-------|--------------|
| High     | 0     | 0     | 0            |
| Medium   | 1     | 0     | 1            |
| Low      | 2     | 2     | 0            |

Centralization score



Signature



Digitally signed by Trust  
Date: 2025.11.23 19:24:19 +02'00'

|                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------|----|
| EXECUTIVE SUMMARY                                                                                            | 1  |
| DOCUMENT PROPERTIES                                                                                          | 4  |
| Versioning                                                                                                   | 4  |
| Contact                                                                                                      | 4  |
| INTRODUCTION                                                                                                 | 5  |
| Scope                                                                                                        | 5  |
| Repository details                                                                                           | 6  |
| About Trust Security                                                                                         | 6  |
| About the Auditors                                                                                           | 6  |
| Disclaimer                                                                                                   | 7  |
| Methodology                                                                                                  | 7  |
| QUALITATIVE ANALYSIS                                                                                         | 8  |
| FINDINGS                                                                                                     | 9  |
| Medium severity findings                                                                                     | 9  |
| TRST-M-1: Rounding up of user debt leads to bad debt realization beyond vested assets                        | 9  |
| Low severity findings                                                                                        | 10 |
| TRST-L-1: OracleManager functions can break zero price invariant due to exchange rate conversion             | 10 |
| TRST-L-2: LiquidityManagerIsolated._hypotheticalLiquidityOf() fails to check error code for debt token price | 11 |
| Additional recommendations                                                                                   | 12 |
| TRST-R-1: AuctionManager incorrectly documents requireFulfillment flag                                       | 12 |
| TRST-R-2: MIN_LOAN_SIZE is checked incorrectly when there is more than one debt token                        | 12 |
| TRST-R-3: Reentrancy guard modifier should be executed before other modifiers                                | 12 |
| TRST-R-4: BorrowableCToken.flashLoan() uses ineffective assetsHeld() check                                   | 12 |
| TRST-R-5: Derivation of MIN_LIQUIDATION_BUFFER is inconsistent                                               | 13 |
| TRST-R-6: Documentation errors                                                                               | 13 |
| TRST-R-7: Redundant code                                                                                     | 14 |
| TRST-R-8: Missing access controls for non-view functions                                                     | 14 |
| TRST-R-9: MarketManagerIsolated.updateTokenConfig() fails to check closeFactorMin                            | 14 |
| TRST-R-10: Rounding up user debts can cause debtCap to be exceeded                                           | 14 |
| TRST-R-11: StaticPriceAggregator.getRoundData() should match Chainlink feeds                                 | 14 |
| Centralization risks                                                                                         | 16 |
| TRST-CR-1: Emergency Council                                                                                 | 16 |
| TRST-CR-2: DAO                                                                                               | 16 |

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| TRST-CR-3: DAO Timelock                                                         | 17        |
| TRST-CR-4: Risk Manager contracts with market permissions                       | 17        |
| TRST-CR-5: Auction permissions                                                  | 17        |
| <b>Systemic risks</b>                                                           | <b>19</b> |
| TRST-SR-1: Token risks and assumptions                                          | 19        |
| TRST-SR-2: Oracle risks                                                         | 19        |
| TRST-SR-3: Min/max price guards can override oracle prices and block operations | 20        |
| TRST-SR-4: Extreme interest risk and interest assumptions                       | 20        |
| TRST-SR-5: External protocol risk                                               | 20        |
| TRST-SR-6: Dual borrowable market setup can disincentivize liquidators          | 21        |

# Document properties

## Versioning

| Version | Date       | Description       |
|---------|------------|-------------------|
| 0.1     | 22/11/2025 | Client Report     |
| 0.2     | 23/11/2025 | Mitigation Review |

## Contact

**Trust**

trust@trust-security.xyz

# Introduction

Trust Security has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

## Scope

Curvance contracts repository:

- contracts/architecture/CentralRegistry.sol
- contracts/architecture/DAOTimelock.sol
- contracts/calldata-checker/BaseCalldataChecker.sol
- contracts/calldata-checker/multicall-checker/BaseMulticallChecker.sol
- contracts/calldata-checker/multicall-checker/RedstoneAdaptorMulticallChecker.sol
- contracts/calldata-checker/swap-checker/BaseSwapChecker.sol
- contracts/calldata-checker/swap-checker/OdosV2CalldataChecker.sol
- contracts/libraries/ActionRegistry.sol
- contracts/libraries/Bytes32Helper.sol
- contracts/libraries/CentralRegistryLib.sol
- contracts/libraries/CommonLib.sol
- contracts/libraries/ConstantsLib.sol
- contracts/libraries/LowLevelCallsHelper.sol
- contracts/libraries/Multicall.sol
- contracts/libraries/PluginDelegable.sol
- contracts/libraries/ReentrancyGuardTransient.sol
- contracts/libraries/RescueLib.sol
- contracts/libraries/SwapperLib.sol
- contracts/market/DynamicIRM.sol
- contracts/market/isolated/LiquidityManagerIsolated.sol
- contracts/market/isolated/MarketManagerIsolated.sol
- contracts/market/position-management/BasePositionManager.sol
- contracts/market/position-management/NativeVaultPositionManager.sol
- contracts/market/position-management/SimplePositionManager.sol
- contracts/market/position-management/VaultPositionManager.sol
- contracts/market/token/BaseCToken.sol
- contracts/market/token/BaseCTokenWithYield.sol
- contracts/market/token/BorrowableCToken.sol
- contracts/oracles/adaptors/BaseOracleAdaptor.sol
- contracts/oracles/adaptors/chainlink/ChainlinkAdaptor.sol
- contracts/oracles/adaptors/redstone/RedstoneClassicAdaptor.sol
- contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol
- contracts/oracles/adaptors/wrappedAggregators/BaseWrappedAggregator.sol

- contracts/oracles/adaptors/wrappedAggregators/VaultAggregator.sol
- contracts/oracles/adaptors/wrappedAggregators/WstETHAggregator.sol
- contracts/oracles/adaptors/wrappedAggregators/StaticPriceAggregator.sol
- contracts/oracles/OracleManager.sol
- contracts/plugins/BaseZapper.sol
- contracts/plugins/market/NativeVaultZapper.sol
- contracts/plugins/market/SimpleZapper.sol
- contracts/plugins/market/VaultZapper.sol

Atlas integration repository:

- src/AuctionManager.sol

## Repository details

- **Curvance contracts repository:** <https://github.com/curvance/curvance-contracts>
- **Curvance contracts commit hash:** f10667cc05e3a57b2a63965fa20fc9953b521923
- **Curvance contracts mitigation hash:**  
6ea8779641e9b77e50691eae74ab3330d727b1cf
- **Curvance contracts second mitigation hash:**  
1afe63a47faee5239e924cb135c6b803e63a9154
- **Atlas integration repository:** <https://github.com/curvance/Atlas-Integration>
- **Atlas integration commit hash:** 48e5a916d7392234f75999d601256d807f244d1a
- **Atlas integration mitigation hash:** 98bb9383f80dcacc13c40a40884fbfe8d583d683

## About Trust Security

Trust Security has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

## About the Auditors

Trust has established a dominating presence in the smart contract security ecosystem since 2022. He is a resident on the Immunefi, Sherlock and C4 leaderboards and is now focused in auditing and managing audit teams under Trust Security. When taking time off auditing & bug hunting, he enjoys assessing bounty contests in C4 as a Supreme Court judge.

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at Trust Security and Senior Watson at Sherlock.

ether\_sky is a security researcher with a focus on blockchain security. He specializes in algorithms and data structures and has a solid background in IT development. He has placed at the top of audit contests in Code4rena and Sherlock recently.

## Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. Trust Security assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

## Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

## Qualitative analysis

| Metric               | Rating    | Comments                                                                                |
|----------------------|-----------|-----------------------------------------------------------------------------------------|
| Code complexity      | Good      | Code is mostly simple and well structured, reducing attack risks.                       |
| Documentation        | Excellent | Project is well documented.                                                             |
| Best practices       | Good      | Project generally follows best practices.                                               |
| Centralization risks | Good      | Project is managed by trusted roles, which can be set up with fine grained permissions. |

# Findings

## Medium severity findings

TRST-M-1: Rounding up of user debt leads to bad debt realization beyond vested assets

- **Category:** Rounding issues
- **Source:** BorrowableCToken.sol
- **Status:** Acknowledged

### Description

*BorrowableCToken* implements a rounding policy that rounds against borrowers and it is known that due to rounding up user debts, **marketOutstandingDebt** can be less than the sum of all user debts. Specifically, **marketOutstandingDebt** represents a lower bound for the sum of debt that users owe the protocol. Since **\_totalAssets** and **marketOutstandingDebt** are incremented by the same **assetsToVest** variable in [accrueIfNeeded\(\)](#), the accumulated rounding errors of rounded up debt are also not part of **\_totalAssets**.

To rescue otherwise unrecoverable assets, *BorrowableCToken* implements a *skim()* function that lets the DAO withdraw any asset balance beyond **\_totalAssets**. However, the flipside of repayment is the realization of bad debt. If the accumulated rounding errors are realized as bad debt, then **\_totalAssets** is decremented. This means users are exposed to the risk that accumulated rounding errors are realized as bad debt but do not benefit from repayment of this debt. Stated differently, user debts consist of (1) debt that is accounted for in **\_totalAssets**, i.e., that was part of an accrual of **assetsToVest**, and (2) debt that is not accounted for in **\_totalAssets**, i.e., debt created due to rounding up of user debts. Users face an asymmetric exposure to (2). There is a risk of loss but not benefit if the debt is repaid. Another way to view it is that the rounding direction choice against borrowers in **marketDebtIndex** is at the same time a choice to round against creditors, in bad debt scenarios. The total loss is capped by the number of borrowers multiplied by the number of accruals, which under the stated token assumptions could be significant value.

### Recommended mitigation

Since it is not easily possible to calculate the fractions of repaid debt that constitute either (1) or (2), a refactoring of the debt accounting logic is necessary to solve this issue of asymmetric exposure to debt rounding errors. If the issue is determined to be an acceptable downside of how debt accounting is implemented, it may be acknowledged.

### Team response

Acknowledged.

## Low severity findings

TRST-L-1: OracleManager functions can break zero price invariant due to exchange rate conversion

- **Category:** Logical issues
- **Source:** OracleManager.sol
- **Status:** Fixed

### Description

*OracleManager* aims to maintain an invariant that `price==0` implies `errorCode==BAD_SOURCE`. This invariant can be violated if the source of `price==0` is multiplication by a cToken exchange rate, e.g., in [getPriceIsolatedPair\(\)](#). In the inner `_getPrice()` function, a price greater than zero may be observed which is [not bubbled up](#) as an error, and only the subsequent multiplication makes it equal to zero. Even though the finding is a violation of an invariant, upstream components do not currently rely on this invariant, and so the finding does not present an immediate risk.

### Recommended mitigation

It's recommended to move the `price==0 && errorCode<BAD_SOURCE` check out of `_getPrice()` and into the outer functions to apply it after the multiplication by the cToken exchange rate. The affected functions are `getPrice()`, `getPriceIsolatedPair()`, `getPricesForMarket()` and `_getNativeUSD()`.

### Team response

Fixed by commit [6ea8779](#).

### Mitigation review

All instances except one have been fixed correctly. In `getPriceIsolatedPair()`, the multiplication by the exchange rate must be moved before the `price==0 && errorCode<BAD_SOURCE` check.

```
collateralSharesPrice = FixedPointMathLib.mulDiv(
    collateralSharesPrice,
    ICToken(collateralToken).exchangeRateUpdated(),
    WAD
);

// If somehow an adaptor returns a price of 0, make sure a BAD_SOURCE
// flag is bubbled up.
if (collateralSharesPrice == 0 && errorCode < BAD_SOURCE) {
    errorCode = BAD_SOURCE;
}

if (errorCode >= errorCodeBreakpoint) {
    revert OracleManager__ErrorCodeFlagged();
}
```

### Team response

Fixed by commit [1afe63a](#).

### Mitigation review

Verified, the recommendation has been implemented.

TRST-L-2: `LiquidityManagerIsolated._hypotheticalLiquidityOf()` fails to check error code for debt token price

- **Category:** Logical issues
- **Source:** `LiquidityManagerIsolated.sol`
- **Status:** Fixed

### Description

For debt tokens that are borrowed for the first time, `_hypotheticalLiquidityOf()` requests the price of `snap.underlying`, i.e., the borrowed debt token.

```
// We can skip the error code check as we've already
// priced the share token which requires pricing the
// underlying token.
(prices[i], ) =
    CommonLib._oracleManager(centralRegistry).
        getPrice(snap.underlying, true, false);
```

The documentation for this code states that the error code does not need to be verified. While it is true that the error code for the price queried with `getLower=true` has been checked, here the price for `getLower=false` is queried. It is theoretically possible, depending on future adaptor implementations, that only one of the prices has an error, and so the liquidity calculation may proceed with an invalid price. This in turn may allow the creation of bad debt or premature liquidation of healthy accounts.

### Recommended mitigation

It is recommended to check the returned error code and to revert if `errorCode >= action.errorCodeBreakpoint`.

### Team response

Fixed by commit [6ea8779](#).

### Mitigation review

Verified, the recommendation has been implemented.

## Additional recommendations

### TRST-R-1: AuctionManager incorrectly documents requireFulfillment flag

Contrary to the [documentation](#) for the *AuctionManager* constructor, setting **requireFulfillment=false** means that oracle updates do proceed even if all solvers fail, and this is intended. The documentation should be updated accordingly.

### TRST-R-2: MIN\_LOAN\_SIZE is checked incorrectly when there is more than one debt token

In *LiquidityManagerIsolated.\_hypotheticalLiquidityOf()*, **MIN\_LOAN\_SIZE** is [checked](#) against the **newDebt** variable which is an accumulator for all debts. Since for now the Curvance contracts only allow at most one debt token, the accumulated debt is equal to the total debt.

However, the logic is inconsistent. If the intention is that for every asset the debt should be above the minimum, then this doesn't hold, because after passing the threshold, any additional assets automatically pass the check. If the intention is that for all assets together the debt should be above the minimum, then the check could fail early before all assets are accounted for.

Assuming the intention is that each asset should pass the minimum check, the check must be applied to each asset's debt separately. This could apply to future market contracts which reuse the *LiquidityManagerIsolated* code.

### TRST-R-3: Reentrancy guard modifier should be executed before other modifiers

Function modifiers are executed left-to-right, which means the *preDeposit()* and *checkSlippage()* modifiers in *BasePositionManager* are executed before the *nonReentrant()* modifier and are not protected against reentrancy. It is not possible for the logic in *preDeposit()* and *checkSlippage()* to return early or to trigger a callback, therefore the protection granted by the *nonReentrant()* modifier is not affected. Still, it is recommended to move the *nonReentrant()* modifier to the leftmost position.

### TRST-R-4: BorrowableCToken.flashLoan() uses ineffective assetsHeld() check

The *flashLoan()* function calls *\_checkAssetsHeld()* to check the loaned **assets** against *assetsHeld()*. This check is ineffective for reentrant calls into *flashLoan()* since *assetsHeld()* is invariant with regards to the asset balance. Thus, *\_checkAssetsHeld()* passes even though no assets are available. Since it is intended that all assets can be used for flash loans, and there is no risk of invariant manipulation in flash loan callbacks, there is no impact to the finding. It is recommended to replace the call to *\_checkAssetsHeld()* with a check of

`_asset.balanceOf(address(this))` which correctly reflects the changing balance in reentrant calls.

#### TRST-R-5: Derivation of `MIN_LIQUIDATION_BUFFER` is inconsistent

In the constructor of *LiquidityManagerIsolated*, [MIN\\_LIQUIDATION\\_BUFFER\\_REQUIRED](#) is set to `AUCTION_BUFFER – EXTRA_BUFFER_BEFORE_LIQUIDATION`. This result is then used inside *MarketManagerIsolated.updateTokenConfig()* in the following check:

```
if (
    inputConfig.collRatio >
    (MIN_LIQUIDATION_BUFFER_REQUIRED * AUCTION_BUFFER / (BPS + inputConfig.collReqSoft))
){
    _revert(_INVALID_PARAMETER_SELECTOR);
}
```

This check, given the derivation of `MIN_LIQUIDATION_BUFFER_REQUIRED`, expands to:

```
if (
    inputConfig.collRatio >
    ((AUCTION_BUFFER - EXTRA_BUFFER_BEFORE_LIQUIDATION) * AUCTION_BUFFER / (BPS + inputConfig.collReqSoft))
){
    _revert(_INVALID_PARAMETER_SELECTOR);
}
```

There is no reason why `collRatio` should depend on the square of `AUCTION_BUFFER`. `MIN_LIQUIDATION_BUFFER` can start from `BPS` rather than `AUCTION_BUFFER` since the `AUCTION_BUFFER` is already accounted for.

#### TRST-R-6: Documentation errors

- `increaseThresholdStart` and `decreaseThresholdEnd` are in `BPS * 10` rather than `BPS`

```
/// @param increaseThresholdStart The utilization rate at which the vertex
///     multiplier will begin to increase,
///     in `BPS`.
/// @param decreaseThresholdEnd The utilization rate at which the vertex
///     multiplier negative velocity will max out,
///     in `BPS`.
```

- Market permissions instead of timelock permissions are required for unpausing. The [wrong documentation](#) applies to the following functions in *MarketManagerIsolated*:
  - `setMintPaused()`
  - `setCollateralizationPaused()`
  - `setBorrowPaused()`

- *setLiquidationPaused()*
- *setRedeemPaused()*
- *setTransferPaused()*

#### TRST-R-7: Redundant code

- All functions in *Bytes32Helper* except [toBytes32\(\)](#) are no longer used and may be removed
- [MarketManagerIsolated.canRepay\(\)](#) is never called since it has been replaced by *canRepayWithReview()*

#### TRST-R-8: Missing access controls for non-view functions

Several functions are not declared as **view** but don't implement sufficient access controls. These functions include *OracleManager.getPricesForMarket()*, *MarketManagerIsolated.canLiquidate()* and *MarketManagerIsolated.canRepayWithReview()*. There is no impact in any of the three cases, and the access control is recommended as a best practice.

In *OracleManager.getPricesForMarket()*, it should be checked that the **cTokens** mapping is not empty before calling *getSnapshotUpdated()*. *MarketManagerIsolated.canLiquidate()* should check that **msg.sender** is **action.debtToken**, and *MarketManagerIsolated.canRepayWithReview()* should validate that **cToken** is listed.

#### TRST-R-9: MarketManagerIsolated.updateTokenConfig() fails to check closeFactorMin

If an auction manager sets their close factor to zero it means they want to use the default parameters, so zero should not also be in the range of [**closeFactorMin**,**closeFactorMax**]. This check is missing from [updateTokenConfig\(\)](#).

#### TRST-R-10: Rounding up user debts can cause debtCap to be exceeded

For each *BorrowableCToken* a **debtCap** is defined which [limits marketOutstandingDebt](#) in *\_canBorrow()*. It should be documented that by rounding up the debt of each user, up to 1 wei for each time a user has their interest accrued, the actual sum of user debts may exceed **marketOutstandingDebt** and thus **debtCap**.

#### TRST-R-11: StaticPriceAggregator.getRoundData() should match Chainlink feeds

To match the implementation of common Chainlink feeds (e.g. [BTC/USD](#)), *getRoundData()* should return the provided input parameter as the first and last return value. Additionally, if

the input round is  $>1$ , **answer**, **startedAt** and **updatedAt** should be zero since **latestRound()** **== 1**.

## Centralization risks

### TRST-CR-1: Emergency Council

The Emergency Council is assigned DAO permissions, elevated permissions, and market permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasDaoPermissions;  
mapping(address => bool) public hasElevatedPermissions;  
mapping(address => bool) public hasMarketPermissions;
```

The Emergency Council is also assigned the canceller role inside *DAOTimelock*:

```
_grantRole(CANCELLER_ROLE, _EMERGENCY_COUNCIL);
```

In contrast to *DAOTimelock*, the Emergency Council is expected to act without a delay. The Emergency Council can configure protocol parameters, pause markets, add or remove assets from oracles, update fees, cancel timelock transactions, and perform any action requiring elevated permissions. The Emergency Council is therefore fully trusted with a trust level even higher than the *DAOTimelock*.

### TRST-CR-2: DAO

The DAO address is assigned DAO permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasDaoPermissions;
```

It is set as the proposer, executor, and canceller inside *DAOTimelock*:

```
_DAO_ADDRESS = cr.daoAddress();  
_grantRole(PROPOSER_ROLE, _DAO_ADDRESS);  
_grantRole(EXECUTOR_ROLE, _DAO_ADDRESS);  
_grantRole(CANCELLER_ROLE, _DAO_ADDRESS);
```

Elevated permissions (DAO Timelock and Emergency Council) can call *CentralRegistry.transferDaoPermissions()* to change the DAO address:

```
function transferDaoPermissions(address newDaoAddress) public virtual {  
    _checkElevatedPermissions();  
  
    //
```

DAO permissions are allowed to directly call *CentralRegistry.withdrawFees()* and *BorrowableCToken.skim()* without timelock delay. The intention is that the DAO cannot perform malicious actions unless going through the **>=5 day** timelock.

## TRST-CR-3: DAO Timelock

The DAO Timelock is assigned DAO permissions, elevated permissions, and market permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasDaoPermissions;  
mapping(address => bool) public hasElevatedPermissions;  
mapping(address => bool) public hasMarketPermissions;
```

The timelock is managed by the DAO address with a timelock delay of at least **5 days**. The Emergency Council can call *CentralRegistry.transferTimelockPermissions()* to change the timelock address:

```
function transferTimelockPermissions(address newTimelock) external {  
    _checkEmergencyCouncilPermissions();  
    //
```

The DAO timelock can configure protocol parameters, manage markets, update oracle settings, adjust fees, and perform administrative functions. By enforcing the minimum delay of 5 days, users are provided with sufficient time for reacting to proposals. Overall, the DAO Timelock is fully trusted but operates under transparency via the timelock mechanism, this means the community can likely react to malicious operations in time.

## TRST-CR-4: Risk Manager contracts with market permissions

Risk Managers are assigned market permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasMarketPermissions;
```

Risk Managers are contracts with their own specific restrictions on actions. These restrictions can be enforced by the external Risk Manager contracts, for example, when calling *setGuardedPriceConfig()*, a restriction may be implemented to only adjust values by a specific percentage per week.

A Risk Manager can have any trust level depending on the Risk Manager contract implementation. For example, updating **collReqSoft** via *updateTokenConfig()* can cause instant liquidations, since positions that were previously healthy under the old configuration can suddenly become liquidatable under the new one.

## TRST-CR-5: Auction permissions

*AuctionManager* contracts are assigned auction permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasAuctionPermissions;
```

*AuctionManager* contracts can call *CentralRegistry.unlockAuctionForMarket()*, *CentralRegistry.resetAuctionForMarket()*,

*MarketManagerIsolated.setTransientLiquidationConfig()*, and *MarketManagerIsolated.resetTransientLiquidationConfig()*. They can set **incentive** and **closeFactor** within the range allowed by the market's configuration:

```
if (incentive < c.liqIncMin || incentive > c.liqIncMax) {
    _revert(_INVALID_PARAMETER_SELECTOR);
}

// Validate 'closeFactor' is within configured allowed close factor
// range. This also validates 'closeFactor' is not > the 16 bits we
// have allocated.
if (closeFactor < c.closeFactorMin || closeFactor > c.closeFactorMax) {
    _revert(_INVALID_PARAMETER_SELECTOR);
}
```

Collateral values for health checks are discounted by **AUCTION\_BUFFER** when liquidated with an *AuctionManager*, granting priority to perform liquidations.

The *AuctionManager* contract itself has **bundler**, **auctioneer**, **user**, and **governor** roles. The **auctioneer** and **user** roles are held by the same Fastlane EOA. The **bundler** needs to be part of the data signed by the Fastlane EOA (Fastlane will be liberal in assigning bundlers), and the **governor** is a multisig with Curvance and Fastlane signers.

Regardless of role assignments, the principal consideration with regards to the safety of Curvance is that liquidatable accounts can be liquidated with **incentive** and **closeFactor** within the allowed range.

## Systemic risks

### TRST-SR-1: Token risks and assumptions

The protocol makes assumptions about token behaviors and properties that are critical for safe operation. These include assumptions that tokens do not have a fee-on-transfer (FoT), are not rebasing, do not perform callbacks, and comply with ERC20. Tokens are expected to have between 6 and 24 decimals.

The protocol trusts tokens not to change their behaviors and to stay within assumed ranges of value. The following token value and range assumptions must hold:

- Minimum \$ price per token = **\$5e-4 (\$0.0005)**, in WAD: **5e14**
- Maximum \$ price per token = **\$10e6 (\$10m)**, in WAD: **1e25**
- Maximum \$/wei: **\$1e7 / 1e8 wei = 0.1 \$ per wei**
- Minimum \$/wei: **\$5e-4 / 1e24 wei = 5e-28 \$ per wei**
- **totalSupply** fits into **uint136** (minimum market cap to reach this is **2<sup>136</sup> wei \* 5e-28 \$ per wei = \$43T**). Market caps up to **\$43T** can safely be used within the protocol

If tokens violate these assumptions the protocol's accounting and liquidation mechanisms may break down, leading to loss of funds. Unlike systems that can remove malicious tokens reactively, many of the protocol's core operations assume these properties hold continuously. Therefore, tokens must be carefully vetted before listing, and the protocol should maintain a whitelist of supported token functionalities to prevent incompatible assets from being added.

### TRST-SR-2: Oracle risks

Oracles are trusted to report correct prices, and to adhere to their stated heartbeats and deviation thresholds.

Besides PUSH-based oracles that update prices when deviation crosses a threshold (e.g., Chainlink), the protocol also uses a PULL-based Redstone oracle which does not automatically update prices at a certain deviation threshold. If prices are not manually updated frequently, and the allowed staleness period is not configured safely, the protocol may operate with prices that deviate significantly from real market values.

For wrapped aggregators, there is additional risk that *convertToAssets()* (the vault exchange rate) does not return the rate at which the free market would value the shares. Potential causes include:

- Withdrawal locking mechanisms (discount for illiquidity can be expected)
- Withdrawal fees that reduce the actual redeemable value
- Available off-chain information that is not yet represented on-chain

This introduces deviations similar to other oracle types. For example, if a vault has announced a loss but the vault shares are still valued at the old `convertToAssets()` rate, borrowers using those shares as collateral may not be properly liquidated, exposing lenders to bad debt.

#### TRST-SR-3: Min/max price guards can override oracle prices and block operations

The protocol's price guard mechanism returns a successful max price when the upper guard is hit, rather than reverting. And the price guard mechanism returns an invalid zero price when the lower guard is hit which bubbles up as a **BAD\_SOURCE** error. This asymmetric behavior is a design choice. The lower guard acts as a circuit breaker, blocking all protocol operations that rely on the token's price. And the upper guard aims to protect against inflated prices. The risk of capping a price is that the price might genuinely deviate to the upside, which e.g., could allow borrowers to take on excessive debt that has a higher market value than the internal valuation calculated by the protocol.

#### TRST-SR-4: Extreme interest risk and interest assumptions

The protocol assumes interest rates are properly managed and that 5-year interest doesn't appreciate the cToken exchange rate by more than **1000x**. Longer time periods or higher interest rates have not been checked for compatibility with core markets. A margin of safety exists and there is no hard limit where something specific breaks after 5 years or more than **1000x** in 5 years.

An example of a limitation is **marketDebtIndex**, defined as **uint80** with a maximum value of approximately **1.2e24**. Since its starting value is **1e18**, a factor of at most **1.2e6** can be multiplied before an unsafe cast occurs that truncates leading bits:

```
shl(_BITPOS_DEBT_INDEX, marketDebtIndex)
```

At a rate of **1000x** in 5 years, an overflow would occur in roughly 10 years. If interest rates are mismanaged or markets operate for extended periods with high utilization, the debt index could eventually overflow, causing incorrect debt calculations and loss of funds.

#### TRST-SR-5: External protocol risk

Curvance integrates with external protocols including DEXs for swaps, ERC4626 vaults as cToken assets, and the Atlas framework (<https://www.atlasevm.com/>) for auction-based liquidations.

This audit has only assessed the specific integration of Odos (<https://docs.odos.xyz/>) and Atlas. Other DEX integrations and vault protocols have not been in scope. An issue in an external integration can lead to loss of funds and the protocol's security depends on the security of these external integrations.

## TRST-SR-6: Dual borrowable market setup can disincentivize liquidators

Curvance markets can be set up with two *BorrowableCToken* where both **cTokens** can be borrowed using the others shares as collateral.

In such a setup, liquidators may have to discount the received liquidated shares due to multiple factors:

- Shares can't be redeemed immediately during high utilization which is an opportunity cost as well as requiring liquidators to take on credit risk of Curvance.
- Unrealized bad debt can effectively make shares less valuable than calculated in the liquidation flow.